L8: Singular Value Decomposition and Low-Rank Approximation

COMP3670/6670
Linear Algebra in Machine Learning
Yun Kuen Cheung, School of Computing, ANU

A recommender system records how users rate items in a potentially very large rectangular table. However, many users have similar preferences. Can we learn shorter representations that capture these patterns, then use them to recommend items?

Lecture 7 used the eigenvectors of a symmetric covariance matrix to choose a low-dimensional representation. *Singular value decomposition* (SVD) works directly with a rectangular matrix. We will use it to approximate a table by a product of two smaller matrices, then interpret their rows as learned representations. Our arguments build on rank, orthonormal coordinates and the spectral theorem. For ratings that have not been observed, we will need a different fitting objective, but the same idea of a low-rank model will remain useful.

SVD describes a rectangular matrix using orthonormal directions

The spectral theorem from Lecture 7 applies to real symmetric square matrices. A user-item table need not even be square: the number of users and the number of items can differ. For a general matrix $\mathbf{A} \in \mathbb{R}^{m\times d}$, an input direction lies in $\mathbb{R}^d$ and its output lies in $\mathbb{R}^m$. SVD uses one orthonormal list in each space, rather than trying to use the same eigenbasis for both.

Theorem 1. Singular value decomposition. Every nonzero matrix $\mathbf{A} \in \mathbb{R}^{m\times d}$ of rank r admits a factorization

$$\mathbf{A} \;=\; \mathbf{U}_r\mathbf{\Sigma}_r\mathbf{V}_r^{\mathsf{T}}, \qquad \mathbf{U}_r \in \mathbb{R}^{m\times r}, \quad \mathbf{\Sigma}_r \in \mathbb{R}^{r\times r}, \quad \mathbf{V}_r \in \mathbb{R}^{d\times r},$$

where

$$\mathbf{U}_r \;=\; \begin{pmatrix} | & & | \\ \mathbf{u}_1 & \cdots & \mathbf{u}_r \\ | & & | \end{pmatrix}, \qquad \mathbf{V}_r \;=\; \begin{pmatrix} | & & | \\ \mathbf{v}_1 & \cdots & \mathbf{v}_r \\ | & & | \end{pmatrix}, \qquad \mathbf{\Sigma}_r \;=\; \operatorname{diag}(\sigma_1, \ldots, \sigma_r).$$

The columns of $\mathbf{U}_r$ and of $\mathbf{V}_r$ are orthonormal, and $\sigma_1 \geq \cdots \geq \sigma_r > 0$. These numbers are the positive singular values; the vectors $\mathbf{u}_i$ and $\mathbf{v}_i$ are the corresponding left singular vectors and right singular vectors. This factorization, retaining only the positive singular values, is a compact SVD. Equivalently,

$$\mathbf{A} \;=\; \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^{\mathsf{T}}.$$

Before studying its mathematical properties, we illustrate how SVD represents users and items in a recommender system.

Example 2. User and item embeddings. Suppose $\mathbf{A} \in \mathbb{R}^{m\times d}$ records ratings from m users for d items, with every rating known. In its compact SVD, row i of $\mathbf{U}_r$ is an r-dimensional embedding of user i: a vector representing that user. Similarly, row j of $\mathbf{V}_r$ is an embedding of item j. Writing these rows as $\mathbf{p}_i^{\mathsf{T}}$ and $\mathbf{q}_j^{\mathsf{T}}$, respectively, matrix multiplication gives

$$a_{ij} \;=\; \mathbf{p}_i^{\mathsf{T}} \boldsymbol{\Sigma}_r \mathbf{q}_j \;=\; \sum_{\ell=1}^{r} \sigma_\ell p_{i\ell} q_{j\ell}.$$

Thus a rating is a *weighted dot product of the user and item embeddings.*

Each coordinate describes a shared latent pattern in the ratings. On coordinate ℓ, matching signs contribute positively to the score, opposite signs contribute negatively and the singular value σ_ℓ weights that contribution. For films, a pattern might reflect a preference for action over romance, although the learned coordinates need not correspond to named genres.

For example, if $r = 2$, $\mathbf{p}_i = (0.6, 0.2)^{\mathsf{T}}$, $\mathbf{q}_j = (0.5, -0.4)^{\mathsf{T}}$ and the singular values are 10 and 2, then the score is $10(0.6)(0.5) + 2(0.2)(-0.4) = 2.84$. The first pattern contributes 3, while the second reduces the score by 0.16. *SVD turns a table of interactions into vector representations whose compatibility explains each entry.* Keeping only the leading coordinates gives shorter embeddings and approximate scores, as we will study next.

The zero matrix needs no nonzero terms, so we focus on $r \geq 1$. Orthonormality gives $\mathbf{U}_r^{\mathsf{T}}\mathbf{U}_r = \mathbf{V}_r^{\mathsf{T}}\mathbf{V}_r = \mathbf{I}_r$. Recall that $\mathbf{U}_r \in \mathbb{R}^{m\times r}$ and $\mathbf{V}_r \in \mathbb{R}^{d\times r}$, i.e., these two matrices are not square in general, so they are not necessarily orthogonal matrices in the sense of Lecture 5. In particular, we cannot generally replace $\mathbf{U}_r\mathbf{U}_r^{\mathsf{T}}$ or $\mathbf{V}_r\mathbf{V}_r^{\mathsf{T}}$ by an identity matrix.

The right and left singular vectors describe corresponding input and output directions. The following proposition makes this relationship precise and connects the SVD of $\mathbf{A}$ to the eigen-decompositions of the symmetric matrices $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ and $\mathbf{A}\mathbf{A}^{\mathsf{T}}$.

Proposition 3. For each singular-vector pair in Theorem 1,

$$\mathbf{A}\mathbf{v}_i \;=\; \sigma_i \mathbf{u}_i, \qquad \mathbf{A}^{\mathsf{T}}\mathbf{u}_i \;=\; \sigma_i \mathbf{v}_i.$$

Their eigenvector equations are

$$\mathbf{A}^{\mathsf{T}}\mathbf{A}\mathbf{v}_i \;=\; \sigma_i^2 \mathbf{v}_i, \qquad \mathbf{A}\mathbf{A}^{\mathsf{T}}\mathbf{u}_i \;=\; \sigma_i^2 \mathbf{u}_i.$$

Both matrices $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ and $\mathbf{A}\mathbf{A}^{\mathsf{T}}$ have positive eigenvalues $\sigma_1^2, \ldots, \sigma_r^2$, counted with multiplicity. Their remaining eigenvalues are zero: $d - r$ for $\mathbf{A}^{\mathsf{T}}\mathbf{A}$ and $m - r$ for $\mathbf{A}\mathbf{A}^{\mathsf{T}}$.

Proof: Expanding the SVD and using orthonormality gives

$$\mathbf{A}\mathbf{v}_i \;=\; \sum_{j=1}^{r} \sigma_j \mathbf{u}_j \langle \mathbf{v}_j, \mathbf{v}_i \rangle \;=\; \sigma_i \mathbf{u}_i, \qquad \mathbf{A}^{\mathsf{T}}\mathbf{u}_i \;=\; \sum_{j=1}^{r} \sigma_j \mathbf{v}_j \langle \mathbf{u}_j, \mathbf{u}_i \rangle \;=\; \sigma_i \mathbf{v}_i.$$

Applying $\mathbf{A}^{\mathsf{T}}$ to the first identity and $\mathbf{A}$ to the second proves the two eigenvector equations. Moreover,

$$\mathbf{A}^{\mathsf{T}}\mathbf{A} \;=\; \mathbf{V}_r \boldsymbol{\Sigma}_r \mathbf{U}_r^{\mathsf{T}} \mathbf{U}_r \boldsymbol{\Sigma}_r \mathbf{V}_r^{\mathsf{T}} \;=\; \mathbf{V}_r \boldsymbol{\Sigma}_r^2 \mathbf{V}_r^{\mathsf{T}}, \qquad \mathbf{A}\mathbf{A}^{\mathsf{T}} \;=\; \mathbf{U}_r \boldsymbol{\Sigma}_r \mathbf{V}_r^{\mathsf{T}} \mathbf{V}_r \boldsymbol{\Sigma}_r \mathbf{U}_r^{\mathsf{T}} \;=\; \mathbf{U}_r \boldsymbol{\Sigma}_r^2 \mathbf{U}_r^{\mathsf{T}}.$$

Complete the orthonormal list $\mathbf{v}_1, \ldots, \mathbf{v}_r$ to an orthonormal basis of $\mathbb{R}^d$. Each added vector is orthogonal to every column of $\mathbf{V}_r$, so the displayed product maps it to zero. This accounts for all d eigenvalues of $\mathbf{A}^{\mathsf{T}}\mathbf{A}$. Similarly, completing $\mathbf{u}_1, \ldots, \mathbf{u}_r$ to an orthonormal basis of $\mathbb{R}^m$ supplies the $m - r$ zero-eigenvalue directions of $\mathbf{A}\mathbf{A}^{\mathsf{T}}$. These completed bases give full eigen-decompositions of the two matrices. □

This explains the connection to familiar tools: obtain the positive eigenvalues λ_i and orthonormal eigenvectors $\mathbf{v}_i$ of $\mathbf{A}^\mathsf{T}\mathbf{A}$, then take $\sigma_i = \sqrt{\lambda_i}$ and $\mathbf{u}_i = \mathbf{A}\mathbf{v}_i/\sigma_i$. The matrix $\mathbf{A}^\mathsf{T}\mathbf{A}$ is symmetric and positive semidefinite because $(\mathbf{A}^\mathsf{T}\mathbf{A})^\mathsf{T} = \mathbf{A}^\mathsf{T}\mathbf{A}$ and $\mathbf{x}^\mathsf{T}\mathbf{A}^\mathsf{T}\mathbf{A}\mathbf{x} = \|\mathbf{A}\mathbf{x}\|_2^2 \geq 0$. These are the same calculations used for covariance in Lecture 7. They explain SVD mathematically; numerical software can compute an SVD directly without explicitly forming $\mathbf{A}^\mathsf{T}\mathbf{A}$.

Example 4. Consider

$$\mathbf{A} = \begin{pmatrix} 20 & 10 \\ 8 & 19 \\ -2 & 14 \end{pmatrix}, \qquad \mathbf{A}^\mathsf{T}\mathbf{A} = \begin{pmatrix} 468 & 324 \\ 324 & 657 \end{pmatrix}.$$

The right-hand matrix has orthonormal eigenvectors

$$\mathbf{v}_1 = \frac{1}{5}\begin{pmatrix} 3 \\ 4 \end{pmatrix}, \qquad \mathbf{v}_2 = \frac{1}{5}\begin{pmatrix} 4 \\ -3 \end{pmatrix}$$

with eigenvalues 900 and 225, respectively. Thus the singular values are 30 and 15. The corresponding left singular vectors are

$$\mathbf{u}_1 = \frac{\mathbf{A}\mathbf{v}_1}{30} = \frac{1}{3}\begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix}, \qquad \mathbf{u}_2 = \frac{\mathbf{A}\mathbf{v}_2}{15} = \frac{1}{3}\begin{pmatrix} 2 \\ -1 \\ -2 \end{pmatrix}.$$

Their squared norms are $(4+4+1)/9 = 1$ and $(4+1+4)/9 = 1$, and their dot product is $(4-2-2)/9 = 0$. Hence a compact SVD is

$$\mathbf{A} = \left[\frac{1}{3}\begin{pmatrix} 2 & 2 \\ 2 & -1 \\ 1 & -2 \end{pmatrix}\right] \begin{pmatrix} 30 & 0 \\ 0 & 15 \end{pmatrix} \left[\frac{1}{5}\begin{pmatrix} 3 & 4 \\ 4 & -3 \end{pmatrix}\right]^\mathsf{T}.$$

For any input $\mathbf{x}$, $\mathbf{A}\mathbf{x} = \mathbf{U}_r\mathbf{\Sigma}_r\mathbf{V}_r^\mathsf{T}\mathbf{x}$ has three stages: extract coordinates along the $\mathbf{v}_i$, scale them by the σ_i and combine the output directions $\mathbf{u}_i$. Since both singular vectors have unit length, σ_i measures how much $\mathbf{A}$ stretches the particular input direction $\mathbf{v}_i$.

Truncation gives a best low-rank approximation

The expansion $\sum_i \sigma_i \mathbf{u}_i \mathbf{v}_i^\mathsf{T}$ separates a matrix into simple contributions. If only a few contributions are large, we may be able to keep a small number while changing the entries only slightly. We first make both "simple contribution" and "changing only slightly" precise.

Definition 5. For nonzero vectors $\mathbf{u} \in \mathbb{R}^m$ and $\mathbf{v} \in \mathbb{R}^d$, their outer product $\mathbf{u}\mathbf{v}^\mathsf{T}$ is the $m \times d$ matrix with entry $u_i v_j$ in position (i, j). It has rank one: every column is a multiple of $\mathbf{u}$, and at least one column is nonzero. The Frobenius norm of $\mathbf{A} \in \mathbb{R}^{m\times d}$ is

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{d} a_{ij}^2}.$$

Thus $\|\mathbf{A} - \mathbf{B}\|_F^2$ is the sum of squared entrywise errors. It is also the sum of the squared Euclidean lengths of the rows of $\mathbf{A} - \mathbf{B}$.

Definition 6. For a matrix with compact SVD $\mathbf{A} = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^{\mathsf{T}}$ and $1 \le k \le r$, its rank-k truncated SVD is

$$\mathbf{A}_k \;=\; \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^{\mathsf{T}} \;=\; \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^{\mathsf{T}},$$

where the subscript k retains the first k columns of each vector matrix and the first k diagonal entries of $\mathbf{\Sigma}_r$.

Every column of $\mathbf{A}_k$ is a combination of $\mathbf{u}_1, \ldots, \mathbf{u}_k$. Conversely, $\mathbf{A}_k \mathbf{v}_i = \sigma_i \mathbf{u}_i$ for $i \le k$, so each of these k independent vectors lies in its column space. Therefore its rank is exactly k. The following theorem says that discarding the smaller singular values is optimal for the squared-error criterion.

Theorem 7. Eckart-Young theorem (1936), squared-error form. For every matrix $\mathbf{B} \in \mathbb{R}^{m \times d}$ with rank at most k,

$$\|\mathbf{A} - \mathbf{B}\|_F^2 \;\ge\; \|\mathbf{A} - \mathbf{A}_k\|_F^2 \;=\; \sum_{i=k+1}^{r} \sigma_i^2.$$

In particular, $\mathbf{A}_k$ is a best approximation among all matrices of rank at most k. When $k = r$, the sum is empty and the reconstruction is exact.

Example 8. For the matrix in Example 4, keeping only the singular value 30 gives

$$\mathbf{A}_1 \;=\; 30 \mathbf{u}_1 \mathbf{v}_1^{\mathsf{T}} \;=\; \begin{pmatrix} 12 & 16 \\ 12 & 16 \\ 6 & 8 \end{pmatrix}, \qquad \mathbf{A} - \mathbf{A}_1 \;=\; \begin{pmatrix} 8 & -6 \\ -4 & 3 \\ -8 & 6 \end{pmatrix}.$$

The squared error is $8^2 + (-6)^2 + (-4)^2 + 3^2 + (-8)^2 + 6^2 = 225$. By Theorem 7, no rank-one matrix gives a smaller squared error.

Storing the three truncated factors takes $k(m + d + 1)$ numbers instead of md. There is a saving only when the chosen number of factors is sufficiently small. Repeated singular values can allow different equally good truncated approximations; the theorem does not promise uniqueness.

Recommender systems use low-dimensional user and item factors

A user's ratings may reflect a few recurring preferences rather than an independent decision for every item. For example, users who enjoy similar kinds of films may rate many films similarly. We begin with an idealized table in which every rating is known. This lets us understand the model before addressing the missing ratings that make recommendation a prediction problem.

Definition 9. A user-item rating matrix $\mathbf{R} \in \mathbb{R}^{n \times m}$ records the ratings of n users for m items. A latent-factor model assigns a vector $\mathbf{p}_i \in \mathbb{R}^k$ to user i and a vector $\mathbf{q}_j \in \mathbb{R}^k$ to item j, and predicts

$$\widehat{r}_{ij} \;=\; \langle \mathbf{p}_i, \mathbf{q}_j \rangle .$$

Stacking the user vectors as rows of $\mathbf{P} \in \mathbb{R}^{n \times k}$ and the item vectors as rows of $\mathbf{Q} \in \mathbb{R}^{m \times k}$ gives

$$\widehat{\mathbf{R}} \;=\; \mathbf{P}\mathbf{Q}^{\mathsf{T}}.$$

Here "latent" means that the factors are not supplied as recorded features; they are learned from the ratings. Their coordinates need not correspond to named genres. Every column of $\mathbf{PQ}^\mathsf{T}$ lies in $\mathrm{Col}(\mathbf{P})$, so this prediction matrix has rank at most k.

For a fully observed table, choose $\mathbf{P} = \mathbf{U}_k\mathbf{\Sigma}_k$ and $\mathbf{Q} = \mathbf{V}_k$ from its SVD. Then $\mathbf{PQ}^\mathsf{T} = \mathbf{R}_k$, so Theorem 7 guarantees the smallest possible total squared reconstruction error among all such k-factor models. The item vectors are rows of $\mathbf{V}_k$, not its columns: a column describes one pattern across all items, whereas a row gives one item's coordinates across the patterns.

Example 10. Six users give scores out of 10 to four films:

$$\mathbf{R} \;=\; \begin{pmatrix} 10 & 8 & 4 & 2 \\ 8 & 10 & 2 & 4 \\ 9 & 9 & 3 & 3 \\ 4 & 2 & 10 & 8 \\ 2 & 4 & 8 & 10 \\ 3 & 3 & 9 & 9 \end{pmatrix}.$$

As in Lecture 4, you can apply Gaussian elimination to obtain a row-echelon form of $\mathbf{R}$, and show that $\mathbf{R}$ has rank 3. Its singular values and singular vectors can be computed using software. One compact SVD is

$$\mathbf{R} \;=\; \underbrace{\begin{pmatrix} \frac{1}{\sqrt{6}} & \frac{1}{\sqrt{6}} & \frac{1}{2} \\ \frac{1}{\sqrt{6}} & \frac{1}{\sqrt{6}} & -\frac{1}{2} \\ \frac{1}{\sqrt{6}} & \frac{1}{\sqrt{6}} & 0 \\ \frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{6}} & \frac{1}{2} \\ \frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{6}} & -\frac{1}{2} \\ \frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{6}} & 0 \end{pmatrix}}_{\mathbf{U}_3} \underbrace{\begin{pmatrix} 12\sqrt{6} & 0 & 0 \\ 0 & 6\sqrt{6} & 0 \\ 0 & 0 & 4 \end{pmatrix}}_{\mathbf{\Sigma}_3} \underbrace{\left[\frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & 1 & -1 \end{pmatrix}\right]}_{\mathbf{V}_3^\mathsf{T}}.$$

The columns of $\mathbf{U}_3$ and $\mathbf{V}_3$ are orthonormal, and the singular values satisfy $12\sqrt{6} > 6\sqrt{6} > 4 > 0$. Each column of $\mathbf{U}_3$ describes a pattern across the six users, while the corresponding column of $\mathbf{V}_3$ describes a pattern across the four items.

Reading across instead, row i of $\mathbf{U}_3$ is a three-dimensional embedding of user i, and row j of $\mathbf{V}_3$ is a three-dimensional embedding of item j. Their coordinates describe that user or item across the three latent patterns. The rating r_{ij} is the weighted dot product of these embeddings, with weights $12\sqrt{6}$, $6\sqrt{6}$ and 4.

Retaining the first two singular values and their corresponding vectors gives $\mathbf{R}_2 = \mathbf{U}_2\mathbf{\Sigma}_2\mathbf{V}_2^\mathsf{T}$. To express this approximation as user-item dot products, absorb the singular values into the user factors by setting $\mathbf{P} = \mathbf{U}_2\mathbf{\Sigma}_2$ and $\mathbf{Q} = \mathbf{V}_2$. Explicitly,

$$\mathbf{P} \;=\; \mathbf{U}_2\mathbf{\Sigma}_2 \;=\; \begin{pmatrix} 12 & 6 \\ 12 & 6 \\ 12 & 6 \\ 12 & -6 \\ 12 & -6 \\ 12 & -6 \end{pmatrix}, \qquad \mathbf{Q} \;=\; \mathbf{V}_2 \;=\; \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & -1 \\ 1 & -1 \end{pmatrix},$$

so the reconstructed rating matrix is

$$\mathbf{R}_2 \;=\; \mathbf{P}\mathbf{Q}^{\mathsf{T}} \;=\; \begin{pmatrix} 9 & 9 & 3 & 3 \\ 9 & 9 & 3 & 3 \\ 9 & 9 & 3 & 3 \\ 3 & 3 & 9 & 9 \\ 3 & 3 & 9 & 9 \\ 3 & 3 & 9 & 9 \end{pmatrix}.$$

For user 1, the reconstructed scores for films 1 and 3 are $12/2 + 6/2 = 9$ and $12/2 - 6/2 = 3$. The model retains the main pattern: the first three users prefer the first two films, whereas the last three users prefer the last two. It discards the smaller differences within these groups. By Theorem 7, the total squared error is

$$\|\mathbf{R} - \mathbf{R}_2\|_F^2 \;=\; 4^2 \;=\; 16.$$

The corresponding root mean squared error (RMSE) is $\sqrt{16/24} = \sqrt{2/3} \approx 0.82$.

The complete-table example illustrates representation and reconstruction. Since every entry is known, it does not demonstrate successful prediction of unseen ratings. Nor does the low-rank approximation prove that the discarded differences are noise. To assess recommendation, we must predict ratings that were not used to fit the model.

Missing ratings require fitting only observed entries

Usually a user has rated only a small fraction of the available items. An unrecorded rating is unknown, not zero. Ordinary SVD requires a numerical value in every position, so it cannot be applied directly to the incomplete table. Filling the blanks with zeros would treat a lack of information as a particular rating. Instead, we keep the latent-factor model and change which discrepancies contribute to its loss.

Definition 11. Let $\Omega \subseteq \{1, \dots, n\} \times \{1, \dots, m\}$ be the set of observed training pairs. A regularized k-factor model fits the user and item vectors by minimizing

$$L(\mathbf{P}, \mathbf{Q}) \;=\; \sum_{(i,j)\in\Omega} \big(r_{ij} - \langle \mathbf{p}_i, \mathbf{q}_j \rangle\big)^2 + \lambda \left(\sum_{i=1}^{n} \|\mathbf{p}_i\|_2^2 + \sum_{j=1}^{m} \|\mathbf{q}_j\|_2^2 \right), \qquad \lambda \geq 0.$$

Only observed training ratings enter the first sum. The squared-norm term is a regularization penalty, with its strength controlled by λ.

The factor count k and penalty strength λ are *hyperparameters*: we choose them separately from the fitted user and item vectors. They control the representation size and the strength of regularization, respectively, and can be selected using validation ratings.

As in the SVM objective from Lecture 6, fitting the data competes with keeping parameter norms small. Regularization can discourage overfitting, but it does not guarantee accurate predictions. Unlike SVD factors, the learned $\mathbf{P}$ and $\mathbf{Q}$ need not have orthonormal columns. If every entry is observed and $\lambda = 0$, truncated SVD provides a global minimizer of this objective for $k \leq \operatorname{rank}(\mathbf{R})$. With missing entries or a positive penalty, that conclusion no longer follows from Theorem 7.

Example 12. Missing ratings are common in practice: most users rate only a small fraction of available items. Suppose we observe only the following entries of the rating matrix $\mathbf{R}$ from Example 10:

$$\begin{pmatrix} 10 & 8 & ? & 2 \\ 8 & ? & 2 & 4 \\ 9 & 9 & 3 & ? \\ ? & 2 & 10 & 8 \\ 2 & 4 & ? & 10 \\ 3 & ? & 9 & 9 \end{pmatrix}.$$

Each question mark is an unknown rating. Fit factors $\mathbf{P} \in \mathbb{R}^{6\times k}$ and $\mathbf{Q} \in \mathbb{R}^{4\times k}$ using the optimization program in Definition 11, with only observed training pairs in Ω. Then estimate each missing rating by $\widehat{r}_{ij} = \langle \mathbf{p}_i, \mathbf{q}_j \rangle$; for example, $\widehat{r}_{13} = \langle \mathbf{p}_1, \mathbf{q}_3 \rangle$ estimates user 1's rating for film 3. The values hidden here are available in the earlier example for illustration, but must not be used to fit the factors.

To select k and λ, a practical starting point is to reserve 10–20% of the observed ratings for validation. For example, a split of 80% for training, 10% for validation and 10% for testing separates fitting, hyperparameter selection and final evaluation. These percentages refer only to known entries, excluding missing ones; the best proportions depend on the amount of data available.

Here there are 18 observed ratings, so holding out two for validation gives $2/18 \approx 11\%$. Select these from the observed entries, for example by random sampling subject to keeping training ratings for every user and item being evaluated. Suppose the selected pairs are $(1, 2)$ and $(4, 3)$, whose known ratings are 8 and 10. Their particular indices have no special role. Exclude these pairs from Ω. For each candidate pair (k, λ), fit a fresh model on the same remaining training ratings and compute its validation RMSE:

$$\mathrm{RMSE}_{\mathrm{val}} \;=\; \sqrt{\frac{(8 - \widehat{r}_{12})^2 + (10 - \widehat{r}_{43})^2}{2}}.$$

Choose the hyperparameters with the smallest validation error, then refit on all available observed ratings to estimate the missing entries. In a larger dataset, use a larger validation set and reserve a separate test set of known ratings, excluded from both fitting and hyperparameter selection, for final evaluation. For this evaluation, refitting may use training and validation ratings but must still exclude the test ratings. Keep some training ratings for every user and item being evaluated so that their factors can be learned. A small training error alone does not show that the missing ratings will be estimated accurately.

For a numerical illustration, a Python implementation of alternating least squares tries $k \in \{1, 2, 3\}$ and $\lambda \in \{0.01, 0.02, 0.05, 0.1, 0.2, 0.5, 1, 2, 5\}$, using the validation pairs above. For each fit, it uses ten random initializations and retains the run with the smallest training objective. Among these candidates, validation selects $k = 2$ and $\lambda = 0.01$, with validation RMSE approximately 1.22. Refitting on all 18 observed ratings gives the following estimates, rounded to two decimal places. The original values from Example 10 are shown only for comparison; they were excluded from fitting and selection.

Missing pair (i, j)	$(1, 3)$	$(2, 2)$	$(3, 4)$	$(4, 1)$	$(5, 3)$	$(6, 2)$
Predicted rating	-1.50	7.33	5.61	-0.41	11.00	4.79
Original rating	4	10	3	4	8	3

The predicted ratings show a positive association with the original ratings, although some individual estimates differ substantially. The objective also imposes no constraint that predictions lie between 0 and 10. With only 18 observed ratings and two validation ratings, the information available for fitting the model and choosing hyperparameters is limited. The main purpose of this example is to demonstrate the fitting and validation procedure.

How can the factors be learned? Fix the item vectors temporarily. Each predicted rating is then a linear function of the user's vector, so fitting that user is a squared-error linear-model problem, as in Lecture 2. With a positive penalty, it also charges for the size of the user vector. We can then exchange the roles of users and items. This gives an *alternating fitting* procedure, often called alternating least squares.

Algorithm 1 Alternating fitting of a rating model

Require: Observed training ratings, factor count k and $\lambda > 0$

1: Initialize the factors with small random values, rather than all zeros
2: **repeat**
3: Fix $\mathbf{Q}$ and fit each $\mathbf{p}_i$ using only user i's observed training ratings
4: Fix $\mathbf{P}$ and fit each $\mathbf{q}_j$ using only item j's observed training ratings
5: **until** the objective changes little or a chosen iteration limit is reached
6: Predict missing ratings by $\widehat{r}_{ij} = \langle \mathbf{p}_i, \mathbf{q}_j \rangle$

Each fitting step minimizes its part of L, including its squared-norm penalty; numerical software solves these smaller quadratic problems. Exact minimization at each step cannot increase L. However, each prediction contains products of two learned parameters, so the joint problem is generally not convex. Different initializations can produce different solutions, and we do not claim that alternating fitting finds a global minimum. Deriving the numerical solvers is outside this lecture's scope.

For a user who has not rated an item, its predicted score can be used to rank that item among the candidates. For a user or item with no training ratings, the model has no data from which to learn its factor vector. This is one instance of the cold-start problem.

For a complete table, truncated SVD optimizes total squared reconstruction error. For incomplete ratings, we fit only observed entries and assess predictions on withheld ratings. *A good matrix approximation is a useful tool for ML, but its mathematical optimality does not by itself guarantee success on the learning task.*

Computing SVD with Python

NumPy's `numpy.linalg.svd` computes an SVD directly. The following code uses the matrix from Example 4:

```
import numpy as np

A = np.array([[20, 10], [8, 19], [-2, 14]], dtype=float)
U, s, Vt = np.linalg.svd(A, full_matrices=False)
print(U)
print(s)   # Singular values: [30. 15.]
print(Vt)

A_reconstructed = U @ np.diag(s) @ Vt
k = 1
A_k = U[:, :k] @ np.diag(s[:k]) @ Vt[:k, :]
```

The columns of `U` are left singular vectors, `s` contains the singular values in descending order and `Vt` stores $\mathbf{V}^{\mathsf{T}}$, not $\mathbf{V}$. Here `np.diag(s)` forms the diagonal matrix of singular values, and `@` denotes matrix multiplication. Thus `A_reconstructed` recovers $\mathbf{A}$ up to floating-point rounding, while `A_k` retains only the first k terms.

For an $m \times d$ matrix, `full_matrices=False` retains $\min(m, d)$ terms, so zero singular values may still be included; the compact SVD omits them. Software may reverse the signs of a corresponding pair of singular vectors without changing their contribution to the matrix.

Case study: predicting missing MovieLens ratings

MovieLens is a widely used collection of movie-rating datasets provided by GroupLens. We use *MovieLens 100K*, a small dataset containing 100,000 ratings from 943 users on 1,682 films. Ratings are integers from 1 to 5.[1] Only about 6.3% of the user–film pairs have recorded ratings, so most entries of the rating matrix are unknown.

The accompanying script `movielens.py` implements the regularized alternating-fitting method from the lecture using NumPy. It downloads the dataset on its first run and caches it for later runs.

The script stores observed ratings as triples (i, j, r_{ij}) rather than creating a matrix with zeros in its unknown positions. It fits the objective in Definition 11, not an SVD of a zero-filled matrix.

The experiment has three stages:

1. Split the observed ratings into 80,000 training, 10,000 validation and 10,000 test ratings. Reserve training entries so that every user and film has at least one training rating. This evaluates prediction for existing users and films, not the cold-start problem.
2. Try $k \in \{5, 10, 15, 20, 25, 30\}$ and $\lambda \in \{0.01, 0.02, 0.05, 0.1, 0.2, 0.5, 1, 2, 5\}$, giving 54 candidate pairs. Each fit uses one reproducible random initialization and 15 alternating sweeps. Choose the pair with the smallest validation RMSE. The fixed iteration budget limits computation but does not guarantee an optimal model.
3. Refit using the 90,000 training and validation ratings, then evaluate on test ratings excluded from fitting and hyperparameter selection. Compare with a simple baseline that always predicts the mean of the ratings used for refitting.

The mean-rating baseline computes the arithmetic mean of all 90,000 ratings used for refitting, then predicts this same value for every user–film pair. It ignores individual users' preferences and differences between films. Test ratings are not used to compute the mean. This provides a simple reference against which to compare the fitted model.

Before evaluation, each dot-product prediction is clipped to the rating interval:

$$\widehat{r}_{ij}^{\,\text{clip}} \;=\; \min\{5, \max\{1, \langle \mathbf{p}_i, \mathbf{q}_j \rangle\}\}.$$

Clipping is applied when computing both validation and test RMSE, without changing the objective used to fit the factors. Since every actual rating lies in $[1, 5]$, clipping cannot increase the prediction error for a fixed model. For the set $\mathcal{T}$ of held-out test pairs, the test RMSE is

$$\text{RMSE}_{\text{test}} \;=\; \sqrt{\frac{1}{|\mathcal{T}|} \sum_{(i,j)\in\mathcal{T}} \left(r_{ij} - \widehat{r}_{ij}^{\,\text{clip}}\right)^2}.$$

A smaller RMSE means smaller squared prediction errors on average. RMSE is expressed in the same units as the ratings. The script prints several predicted and actual test ratings, together with the time spent on validation and refitting.

With the reproducible split and fitting procedure above, validation selects $k = 5$ and $\lambda = 5$, with validation RMSE approximately 0.9153. After refitting, the test RMSE is approximately 0.9305, compared

[1] Dataset and usage terms: `https://grouplens.org/datasets/movielens/100k/`. See also Harper and Konstan (2015), *The MovieLens Datasets: History and Context.*

with 1.1320 for the mean-rating baseline. Without clipping, the same fitted model gives test RMSE 0.9321, so clipping provides a small improvement. In this run, the validation search took about 20 seconds and refitting and testing took about 0.3 seconds; runtime depends on the hardware and numerical libraries.

Although an error of nearly one rating point may seem larger than expected, our simple model is reasonably close to a strong published result. Fan et al. (2024) report a MovieLens 100K RMSE of 0.8767 using *AEMC-NE*, an autoencoder-based neural-network approach rather than the matrix factorization model used here.[2] Our RMSE is only about 0.05 rating points higher. However, their result is an average over 20 random 90/10 training/test splits, with validation ratings held out within the training portion. Our split differs, so this comparison provides context rather than a controlled comparison of the two methods.

Finally, the script lists five highly scored films with no recorded rating from user 1. Films are ranked by their raw model scores, while displayed predictions are clipped to $[1, 5]$. Their true ratings are unknown, so they cannot be used to calculate an error. *Holding out known ratings lets us evaluate missing-rating prediction; predicting genuinely unknown ratings produces candidates for recommendation.* A good rating-prediction score does not by itself establish that the resulting recommendations are useful to users.

[2]Jicong Fan, Rui Chen, Zhao Zhang and Chris Ding (2024), *Neuron-Enhanced AutoEncoder Matrix Completion and Collaborative Filtering: Theory and Practice*, ICLR, Section 5.2 and Table 2.